

Problem Session 3: Process Manipulation and Monitoring

Kiatisak Markmeeshap	67070501005
Tithinan Sobking	67070501018
Worawut Sereethai	67070501040
Chanya Poolketkij	67070501058

1 Commands for manipulating and monitoring processes

1.1 The difference between `top` and `ps`

```
$ ps -ef | head -5          # one snapshot, then it exits
UID      PID    PPID  C  STIME TTY          TIME CMD
root      1      0  2  03:11 ?        00:00:01 /sbin/init
root      2      1  0  03:11 ?        00:00:00 /init
root      6      2  0  03:11 ?        00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --
      pipe-fd 10 --log-truncate
root     49      1  0  03:11 ?        00:00:00 /usr/lib/systemd/systemd-journald

$ top -b -n 1 | head -6    # a live monitor, sampled once
top - 03:12:18 up 18:07,  1 user,  load average: 0.12, 0.09, 0.14
Tasks: 34 total,  1 running, 33 sleeping,  0 stopped,  0 zombie
%Cpu(s):  1.2 us,  1.2 sy,  0.0 ni, 96.4 id,  0.0 wa,  0.0 hi,  1.2 si,  0.0 st
MiB Mem :  8909.6 total,  6440.1 free,   986.0 used,  1678.8 buff/cache
MiB Swap: 3072.0 total,  3072.0 free,    0.0 used.  7923.6 avail Mem
```

Listing 1: `ps` and `top` on the same machine

`ps` takes one snapshot of the process table and exits, so its output can be piped, filtered and saved; `top` is an interactive monitor that repaints every few seconds, sorted by CPU usage, and adds system-wide summaries `ps` never prints — load average, task counts by state, memory totals. So `ps` answers “what is running right now, in a form I can process”, and `top` answers “what is this machine doing, and what is consuming it”.

1.2 The purpose of `nice`, and how to `nice` a process

```
$ sleep 120 &                # started normally
$ nice -n 10 sleep 120 &     # started with a raised niceness
$ ps -o pid,ni,comm -p 1405 -p 1406
  PID  NI  COMMAND
 1405   0  sleep
 1406  10  sleep
$ renice -n 5 -p 1405         # raise it on a running process
1405 (process ID) old priority 0, new priority 5
$ renice -n -5 -p 1405        # lowering it needs privilege
renice: failed to set priority for 1405 (process ID): Permission denied
```

Listing 2: Niceness set at start-up, then changed on a running process

`nice` sets a process’s *niceness*, a scheduling hint from -20 to $+19$ where higher means lower priority. It does not cap or reserve CPU time, only weights the scheduler’s choice when several processes are ready at once. `nice -n 10 <command>` starts one at that niceness; `renice -n 5 -p <pid>` changes one already running. Raising niceness is unprivileged, *lowering* it is not — hence the failure above: a user may make their own work less important, but not more important than everyone else’s, which needs `CAP_SYS_NICE`.

1.3 Killing processes and jobs

```
--- by PID ---
$ sleep 300 &                                # pid 1413
$ kill 1413                                   # SIGTERM (15): asks it to stop
(no such process)
$ sleep 300 &                                # pid 1415
$ kill -9 1415                                # SIGKILL (9): cannot be caught
(no such process)
--- by job number ---
$ jobs
[1]+  Running                  sleep 300 &
$ kill %1                                     # %n names a job, not a process
(no jobs left)
--- by name, several at once ---
$ pgrep -x -P $$ sleep | wc -l
3
$ pkill -x -P $$ sleep                        # only this shell's own children
$ pgrep -x -P $$ sleep | wc -l
0
```

Listing 3: Four ways of naming the target

`kill` sends a signal rather than necessarily killing: the default `SIGTERM` (15) asks the process to stop and can be caught so it can clean up, while `kill -9` sends `SIGKILL`, which cannot be caught and so saves nothing — a last resort, not a default. The target is named four ways above: by PID, by PID with an explicit signal, by *job* number (%1 is the shell's own numbering, not a PID), and by name with `pkill`, which kills every match at once. `pkill` matches the process name as a substring, so `-x` demands the whole name; matching the full command line instead is `-f`.

2 Foreground, background, and job control

2.1 First situation: `./ss1_1.sh` versus `./ss1_1.sh &`

`ss1_1.sh` occupies ten seconds. Following the sheet's NOTE it loops with a one-second sleep and prints each tick rather than sleeping once for ten, so that it is visible whether the shell is blocked waiting for it. One difference from an interactive session is noted in the transcript itself: `bash` prints its `[1] <pid>` launch notice only when interactive, so that line does not appear.

```
1  #!/usr/bin/env bash
2  #
3  # PS3 item 2, first situation: a script that occupies 10 seconds. The sheet's
4  # NOTE suggests looping with a one-second sleep rather than one long sleep, so
5  # that progress stays visible while it runs.
6
7  printf 'ss1_1.sh (pid %s):' "$$"
8  for i in $(seq 1 10); do
9      printf ' tick %d' "$i"
10     sleep 1
11 done
12 printf ' finished\n'
```

Listing 4: Source: `ss1_1.sh`

```

--- 1. in the foreground ---
$ date +%T ; ./ss1_1.sh ; date +%T
03:12:21
ss1_1.sh (pid 1434): tick 1 tick 2 tick 3 tick 4 tick 5 tick 6 tick 7 tick 8 tick 9 tick 10 finished
03:12:31

--- 2. in the background ---
$ date +%T ; ./ss1_1.sh &
03:12:31
ss1_1.sh (pid 1448): tick 1 tick 2
$ jobs
[1]+  Running                  ./ss1_1.sh &
$ ps -o pid,ppid,stat,etime,args -p 1448
    PID    PPID STAT      ELAPSED COMMAND
    1448     1431 S          00:02 bash ./ss1_1.sh
$ wait ; date +%T
    tick 3 tick 4 tick 5 tick 6 tick 7 tick 8 tick 9 tick 10 finished
03:12:42

```

Listing 5: The same script run both ways

The script does the same work and takes the same ten seconds either way; what changes is what the *shell* does meanwhile. In the foreground the shell hands the terminal over and waits — the timestamps either side are ten seconds apart, and in between there was no prompt to type at. With `&` the prompt comes back at once and the job goes into the shell’s table, as `jobs` confirms, so `jobs` and `ps` both run while the script is still counting. Its ticks still land in the same terminal, which is why background output interleaves with whatever is typed next.

2.2 Second situation: `ctrl+z`, `jobs`, `bg` and `fg`

`ss1_2.sh` sleeps for 1000 seconds, long enough to suspend and resume. A script has no keyboard and so cannot press `CTRL+Z`: below, the script is genuinely started in the foreground and a small watchdog sends `SIGTSTP` to its process group a few seconds later — exactly what the terminal driver does when `CTRL+Z` is pressed. The job is therefore a real foreground job being really suspended; only the keypress is stood in for.

```

1  #!/usr/bin/env bash
2  #
3  # PS3 item 2, second situation: long enough to suspend and resume -- 1000
4  # seconds, as the sheet asks.
5
6  echo "ss1_2.sh (pid $$): started, now sleeping 1000 seconds"
7  sleep 1000
8  echo "ss1_2.sh (pid $$): finished"

```

Listing 6: Source: `ss1_2.sh`

```

--- 1. run it, then press CTRL+Z ---
$ ./ss1_2.sh
ss1_2.sh (pid 1468): started, now sleeping 1000 seconds
^Z

--- 2. what CTRL+Z did ---
$ jobs
[1]+  Stopped                  ./ss1_2.sh
$ ps -o pid,stat,args -p 1468
    PID STAT  COMMAND
    1468 T    bash ./ss1_2.sh

--- 3. bg: resume it, in the background ---
$ bg %1
[1]+  ./ss1_2.sh &
$ jobs
[1]+  Running                  ./ss1_2.sh &
$ ps -o pid,stat,args -p 1468
    PID STAT  COMMAND
    1468 S    bash ./ss1_2.sh

--- 4. fg: bring it back to the foreground ---
$ fg %1
./ss1_2.sh
Terminated                  ./ss1_2.sh

$ jobs

```

Listing 7: Suspending the job, then resuming it with `bg` and with `fg`

`CTRL+Z` sends `SIGTSTP` to the foreground job and the shell reports `[1]+ Stopped`. The process has *not* been killed: `ps` shows state `T`, stopped by a job-control signal. It is still in the process table, still holds its memory and open files, and still has its place in the job list — it simply will not be scheduled again until told to continue, which is why `CTRL+Z` is not a way to abandon a program.

`fg` and `bg` both resume it by sending `SIGCONT`, and differ only in who gets the terminal afterwards. `bg` resumes it in the background: the state goes from `T` back to `S`, `jobs` reports `Running`, and the prompt stays ours — as though it had been started with `&` in the first place. (`S` is *sleeping*, not running on a CPU: the script is blocked in `sleep(1000)`. What changed is that it is back under the scheduler rather than held off it.) `fg` instead hands the terminal back and blocks until the job ends, exactly like the first run of situation one — so the pair is what makes `CTRL+Z` useful rather than merely disruptive: suspend, move to the background, reclaim the terminal, without restarting the work.

3 Simulating STCF

STCF — Shortest Time-to-Completion First — is the preemptive form of shortest-job-first: the CPU is re-assigned at every time unit, so a process that arrives with less work remaining than the one running takes it away immediately.

Three ways to simulate it, for n processes over a timeline of length T — the sum of the bursts, which is unrelated to n :

- **Linear array, tick by tick** — $O(T \cdot n)$. Step one time unit at a time, scanning all n for the least remaining. Simple, but the same scan is repeated T times.
- **Tick by tick with a min-heap** — $O(T + n \log n)$. A heap ordered by (remaining, pid) makes the process to run the root, and decrementing the root keeps it the root, so a tick costs $O(1)$ and only arrivals and completions disturb the heap. Still tied to T .
- **Event-driven with a min-heap** — $O(n \log n)$; **used here**. The schedule can only change when a process arrives or the running one finishes, so the root is run straight through to the next such event rather than re-examined every unit. At most $2n$ heap operations, and no dependence on how long the bursts are.

Printing follows the same idea: a slot is emitted only when the owner changes, which merges the output into ranges and produces the IDLE gaps. Only the third of the three was implemented; the other two are described above for comparison and are not in the file. `simulate_stfc()` and the min-heap and slot helpers it calls were written for this task, while the includes, the `Process` struct and `main()` came with the skeleton.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAX_PROCESSES 100
6
7 // you may change this if needed
8 typedef struct {
9     int pid;           // process id (int)
10    int arrival;        // arrival time
11    int burst;          // burst time
12    int remaining;      // remaining burst time
13    int finished;       // 0 = not finished, 1 = done
14 } Process;
15
16
17 /* A negative pid means the CPU had nothing to run. */
18 static void print_slot(int pid, int from, int to) {
19     if (pid < 0)
20         printf("[%d-%d]: IDLE\n", from, to);
21     else
22         printf("[%d-%d]: P%d\n", from, to, pid);
23 }
24
25 /* The stretch being printed, held open until somebody else takes the CPU so
26    * that consecutive stretches with the same owner merge into one slot. */
27 typedef struct {
28     int pid;
29     int from;
30     int open;
31 } Slot;
32
33 static void slot_take(Slot *s, int pid, int at) {
34     if (s->open && s->pid == pid)
35         return;           /* same owner: keep extending */
36     if (s->open)
37         print_slot(s->pid, s->from, at);
38     s->pid = pid;
39     s->from = at;
40     s->open = 1;
```

```

41 }
42
43 static void slot_end(Slot *s, int at) {
44     if (s->open)
45         print_slot(s->pid, s->from, at);
46     s->open = 0;
47 }
48
49 /* Ready queue: a min-heap of indices into procs[], ordered by
50  * (remaining, pid) -- the sheet's rule, so the next process to run is the
51  * root. Indices, not copies, so a remaining time is only ever recorded once. */
52 typedef struct {
53     int v[MAX_PROCESSES];
54     int n;
55 } Heap;
56
57 static int runs_first(const Process *p, int a, int b) {
58     if (p[a].remaining != p[b].remaining)
59         return p[a].remaining < p[b].remaining;
60     return p[a].pid < p[b].pid;
61 }
62
63 static void heap_push(Heap *h, const Process *p, int idx) {
64     int i = h->n++;
65     h->v[i] = idx;
66     while (i > 0) { /* sift up */
67         int parent = (i - 1) / 2;
68         if (!runs_first(p, h->v[i], h->v[parent]))
69             break;
70         int tmp = h->v[i]; h->v[i] = h->v[parent]; h->v[parent] = tmp;
71         i = parent;
72     }
73 }
74
75 static int heap_pop(Heap *h, const Process *p) {
76     int top = h->v[0];
77     h->v[0] = h->v[--h->n];
78     int i = 0;
79     for (;;) { /* sift down */
80         int l = 2 * i + 1, r = 2 * i + 2, best = i;
81         if (l < h->n && runs_first(p, h->v[l], h->v[best])) best = l;
82         if (r < h->n && runs_first(p, h->v[r], h->v[best])) best = r;
83         if (best == i)
84             break;
85         int tmp = h->v[i]; h->v[i] = h->v[best]; h->v[best] = tmp;
86         i = best;
87     }
88     return top;
89 }
90
91 static int by_arrival(const void *a, const void *b) {
92     const Process *x = (const Process *)a, *y = (const Process *)b;
93     if (x->arrival != y->arrival)
94         return x->arrival < y->arrival ? -1 : 1;
95     return x->pid < y->pid ? -1 : (x->pid > y->pid);
96 }
97
98 /*
99  * STCF -- Shortest Time-to-Completion First, preemptive.
100  *
101  * The schedule can only change when a process arrives or the running one
102  * finishes, so the chosen process is run through to whichever comes first
103  * instead of re-deciding at every time unit: O(n log n), not O(T*n).
104  *
105  * procs[] is sorted in place; the caller's order is not preserved.
106  */
107 void simulate_stfc(Process *procs, int n) {
108     if (n <= 0)
109         return;
110
111     qsort(procs, (size_t)n, sizeof(procs[0]), by_arrival);
112
113     /* A burst of zero is done before it starts. Left unfinished it would always

```

```

114  * look like the shortest job, be chosen every time, and never end. */
115  int done = 0;
116  for (int i = 0; i < n; i++) {
117      if (procs[i].remaining <= 0 && !procs[i].finished) {
118          procs[i].finished = 1;
119          done++;
120      }
121  }
122
123  Heap ready;
124  ready.n = 0;
125  Slot slot = { -1, 0, 0 };
126  int next = 0; /* the next process to arrive */
127  int t = 0;
128
129  while (done < n) {
130      while (next < n && procs[next].arrival <= t) {
131          if (!procs[next].finished)
132              heap_push(&ready, procs, next);
133          next++;
134      }
135
136      if (ready.n == 0) { /* idle through to the next arrival */
137          if (next >= n)
138              break; /* unreachable, but never hang */
139          slot_take(&slot, -1, t);
140          t = procs[next].arrival;
141          continue;
142      }
143
144      int cur = heap_pop(&ready, procs);
145      slot_take(&slot, procs[cur].pid, t);
146
147      int run = procs[cur].remaining;
148      if (next < n && procs[next].arrival - t < run)
149          run = procs[next].arrival - t; /* stop for the arrival */
150
151      t += run;
152      procs[cur].remaining -= run;
153
154      if (procs[cur].remaining > 0) {
155          heap_push(&ready, procs, cur); /* interrupted, not finished */
156      } else {
157          procs[cur].finished = 1;
158          done++;
159      }
160  }
161
162  slot_end(&slot, t);
163 }
164
165 int main(int argc, char *argv[]) {
166     if (argc < 2) {
167         printf("Usage: %s <csvfile>\n", argv[0]);
168         return 1;
169     }
170
171     FILE *fp = fopen(argv[1], "r");
172     if (!fp) {
173         perror("Error opening file");
174         return 1;
175     }
176
177     Process procs[MAX_PROCESSES];
178     int n = 0;
179     char line[256];
180
181     // skip header line
182     fgets(line, sizeof(line), fp);
183
184     // read each row
185     // you may change this if needed
186     while (fgets(line, sizeof(line), fp) && n < MAX_PROCESSES) {

```

```

187     int pid, arrival, burst;
188     if (sscanf(line, "%d,%d,%d", &pid, &arrival, &burst) == 3) {
189         procs[n].pid = pid;
190         procs[n].arrival = arrival;
191         procs[n].burst = burst;
192         procs[n].remaining = burst;
193         procs[n].finished = 0;
194         n++;
195     }
196 }
197 fclose(fp);
198
199 simulate_stfc(procs, n);
200
201 return 0;
202 }

```

Listing 8: Source: PS3.c

```

void simulate_stfc(Process *procs, int n) {
    if (n <= 0)
        return;

    qsort(procs, (size_t)n, sizeof(procs[0]), by_arrival);

    /* A burst of zero is done before it starts. Left unfinished
    it would always
    * look like the shortest job, be chosen every time, and
    never end. */
    int done = 0;
    for (int i = 0; i < n; i++) {
        if (procs[i].remaining <= 0 && !procs[i].finished) {
            procs[i].finished = 1;
            done++;
        }
    }

    Heap ready;
    ready.n = 0;
    Slot slot = { -1, 0, 0 };
    int next = 0;
    /* the next process to arrive */
    int t = 0;

    while (done < n) {
        while (next < n && procs[next].arrival <= t) {
            if (!procs[next].finished)
                heap_push(&ready, procs, next);
            next++;
        }

        if (ready.n == 0) {
            /* idle through to the next arrival */
            if (next >= n)
                break;
            /* unreachable, but never hang */
            slot_take(&slot, -1, t);
            t = procs[next].arrival;
            continue;
        }

        int cur = heap_pop(&ready, procs);
        slot_take(&slot, procs[cur].pid, t);

        int run = procs[cur].remaining;
        if (next < n && procs[next].arrival - t < run)
            run = procs[next].arrival - t;
        /* stop for the arrival */

        t += run;
        procs[cur].remaining -= run;

        if (procs[cur].remaining > 0) {
            heap_push(&ready, procs, cur);
        }
        /* interrupted, not finished */
        else {
            procs[cur].finished = 1;
            done++;
        }
    }

    slot_end(&slot, t);
}

```

Figure 1: The `simulate_stfc()` function

```

$ ./PS3 case1.csv
[0-2]: P1
[2-4]: P2
[4-5]: P3
[5-7]: P2
[7-12]: P1

$ ./PS3 case2.csv
[0-2]: P1
[2-5]: IDLE
[5-6]: P2
[6-7]: P4
[7-9]: P2
[9-12]: P3

$ ./PS3 case3.csv
[0-3]: P2
[3-5]: P1
[5-7]: P2
[7-8]: P3

```

Listing 9: All three cases

```

kiatisakk@KIATISAK-PC:/mnt/c/Users/kiati/Documents/CPE333-05/PS03$ gcc -std=c11 -Wall -Wextra -o PS3 PS3.c
kiatisakk@KIATISAK-PC:/mnt/c/Users/kiati/Documents/CPE333-05/PS03$ ./PS3 case1.csv
[0-2]: P1
[2-4]: P2
[4-5]: P3
[5-7]: P2
[7-12]: P1
kiatisakk@KIATISAK-PC:/mnt/c/Users/kiati/Documents/CPE333-05/PS03$ ./PS3 case2.csv
[0-2]: P1
[2-5]: IDLE
[5-6]: P2
[6-7]: P4
[7-9]: P2
[9-12]: P3
kiatisakk@KIATISAK-PC:/mnt/c/Users/kiati/Documents/CPE333-05/PS03$ ./PS3 case3.csv
[0-3]: P2
[3-5]: P1
[5-7]: P2
[7-8]: P3
kiatisakk@KIATISAK-PC:/mnt/c/Users/kiati/Documents/CPE333-05/PS03$

```

Figure 2: Running the three provided cases

Discussion

Two assumptions from the sheet shape the output. Because the scheduler’s own work and a context switch cost nothing, the slots abut exactly: a process preempted at $t = 2$ hands over at $t = 2$ and the next slot begins at $t = 2$, where a real machine would show a small gap. And a process arriving at t competes for slot t itself — in the code the `skip procs[i].arrival > t` is evaluated *before* the CPU is assigned for that unit.

Case 1 — preemption. P1 starts alone; at $t = 2$ P2 arrives needing 4 against P1’s remaining 5 and takes the CPU, then at $t = 4$ P3 arrives needing 1 and takes it from P2 in turn. P1 arrived first and finishes last, which is exactly what STCF trades away: short jobs are served quickly at the cost of delaying long ones. Note P2 appears twice, as [2-4] and [5-7] — being preempted does not discard the work already done.

Case 2 — idle time and simultaneous arrivals. P1 finishes at $t = 2$ and nothing arrives until $t = 5$, so the schedule records [2-5]: IDLE: a gap in the arrivals is a gap in the schedule. P2 and P3 then arrive together with identical bursts, and the tie is broken on `pid` so P2 runs.

Case 3 — ties on remaining time. This tests the second tie-break, which neither of the others reaches: a scheduler that got it backwards would still pass case 1. At $t = 3$ the running P2 has 2 left and the arriving P1 needs 2 — equal, so the lower `pid` wins and P1 preempts. At $t = 6$ it repeats the other way round: P2 has 1 left and P3 arrives needing 1, but this time the lower `pid` is the process already running, so P2 keeps the CPU.